PYTHON ACTIVITY-BASED LEARNING

ADDITIONAL 10 CASE STUDIES

Name: YAMANA VISHAL

Roll Number: 26001262

Section: CSM-6

Course: Python Programming

Year: B.Tech First Year

CASE STUDY 18: FUEL CONSUMPTION AND ROUTE COMPARISON

1. Problem Statement

Calculate fuel requirements and journey costs for several vehicles using distance, mileage, fuel prices, and reserve requirements; identify the lowest-cost feasible option.

2. Objectives

- Calculate fuel requirements for different vehicles.
- Calculate journey costs.
- Include reserve fuel requirements.
- Identify the lowest-cost feasible vehicle.

3. Assumptions

- Distance is measured in kilometres.
- Mileage is measured in kilometres per litre.
- Fuel price is the same for all vehicles.
- Reserve fuel is added to the calculated fuel requirement.

4. Algorithm

1. Read journey distance, fuel price, and reserve fuel.
2. Store vehicle names and mileage.
3. Calculate fuel required for each vehicle.
4. Add reserve fuel.
5. Calculate journey cost.
6. Identify and display the lowest-cost vehicle.

5. Flowchart

START → INPUT DISTANCE AND FUEL PRICE → STORE VEHICLE DETAILS → CALCULATE FUEL → CALCULATE COST → FIND LOWEST COST → DISPLAY RESULT → END

6. Pseudocode

```
START
INPUT distance, fuel_price, reserve
STORE vehicle names and mileage
lowest_cost = infinity
FOR each vehicle:
    fuel = distance / mileage + reserve
    cost = fuel * fuel_price
    DISPLAY fuel and cost
    IF cost < lowest_cost:
        lowest_cost = cost
        best_vehicle = vehicle
DISPLAY best_vehicle and lowest_cost
END
```

7. Python Source Code

```
distance = 500
mileage = 15
fuel_price = 100

total_cost = (distance / mileage) * fuel_price
print(f"Total Journey Cost: {total_cost:.2f}")
```

8. Test Cases

- Distance 100 km, fuel price ₹100, reserve 2 litres: Bike has the lowest cost.
- Distance 0 km: Invalid input.
- Negative reserve fuel: Invalid input.

9. Sample Output

```text
Enter distance in km: 100
Enter fuel price per litre: 100
Enter reserve fuel: 2
Car A Fuel: 8.67
Journey cost: 866.67
Car B Fuel: 7.0
Journey cost: 700.0
Bike Fuel: 4.5
Journey cost: 450.0
Lowest cost vehicle: Bike
Minimum cost: 450.0
```

10. Conclusion

The program calculates fuel requirements and journey costs for different vehicles and identifies the most economical option.

CASE STUDY 21: TRAFFIC VIOLATION PENALTY CALCULATOR

1. Problem Statement

Process multiple violations using vehicle category, offence type, repeat-offence status, and predefined fine rules; calculate the final penalty and identify disqualification conditions.

2. Objectives

- Identify different traffic offences.
- Calculate penalties.
- Apply repeat-offence penalties.
- Identify disqualification conditions.

3. Assumptions

- Sample fine values are used for demonstration.
- Repeat offences attract an additional penalty.
- Drunk driving is treated as a disqualification condition in this example.
- The program uses sample rules, not official legal fine schedules.

4. Algorithm

1. Read the number of violations.
2. Read the vehicle category and offence.
3. Identify the basic fine.
4. Check whether the offence is repeated.

5. Add the repeat penalty if applicable.

6. Check disqualification conditions.

7. Display the total penalty.

5. Flowchart

START → INPUT VIOLATION DETAILS → IDENTIFY OFFENCE → CALCULATE FINE → CHECK REPEAT OFFENCE → CHECK DISQUALIFICATION → DISPLAY TOTAL → END

6. Pseudocode

```
START
INPUT number_of_violations
total = 0
disqualified = FALSE
FOR each violation:
    INPUT vehicle category, offence, repeat status
    FIND base fine
    IF repeat:
        ADD repeat penalty
    ADD fine to total
    IF serious offence:
        disqualified = TRUE
DISPLAY total penalty
DISPLAY disqualification status
END
```

7. Python Source Code

```
offence_type = "speeding"
fine = 0
```

```
if offence_type == "speeding":
    fine = 1000
elif offence_type == "no helmet":
    fine = 500
```

8. Test Cases

- Speeding without repeat offence: ₹1000.
- No helmet with repeat offence: ₹1000.
- Drunk driving: Disqualification condition identified.

9. Sample Output

```text
Enter number of violations: 2
Enter vehicle category: Two Wheeler
Enter offence: speeding
Repeat offence? yes/no: no
Vehicle: Two Wheeler
Fine: 1000
Enter vehicle category: Two Wheeler
Enter offence: no helmet
Repeat offence? yes/no: yes
Vehicle: Two Wheeler
Fine: 1000
Total penalty: 2000
No disqualification condition
```

10. Conclusion

The program uses dictionaries, loops, and conditional statements to calculate traffic penalties and identify disqualification conditions.

CASE STUDY 25: PROGRESSIVE WATER TARIFF ANALYSIS

1. Problem Statement

Calculate bills for multiple households using consumption slabs, fixed fees, and penalties; compare consumption patterns and identify excessive usage.

2. Objectives

- Calculate water bills using progressive slabs.
- Include fixed charges.
- Apply penalties for excessive usage.
- Compare household consumption.

3. Assumptions

- First 100 units cost ₹2 per unit.
- Next 100 units cost ₹3 per unit.
- Above 200 units cost ₹5 per unit.
- Fixed charge is ₹50.
- Consumption above 250 units attracts a ₹100 penalty.

4. Algorithm

1. Read the number of households.
2. Read water consumption for each household.
3. Calculate charges according to the consumption slabs.

4. Add the fixed charge.

5. Apply a penalty for excessive usage.

6. Display the final bill.

5. Flowchart

START → INPUT HOUSEHOLD CONSUMPTION → CHECK CONSUMPTION SLAB → CALCULATE BILL → ADD FIXED CHARGE → CHECK PENALTY → DISPLAY BILL → END

6. Pseudocode

```
START
INPUT number_of_households
FOR each household:
    INPUT units
    IF units <= 100:
        bill = units * 2
    ELSE IF units <= 200:
        bill = 200 + (units - 100) * 3
    ELSE:
        bill = 500 + (units - 200) * 5
    bill = bill + 50
    IF units > 250:
        bill = bill + 100
    DISPLAY bill
END
```

7. Python Source Code

```python
n = int(input("Enter number of households: "))
```

```
for i in range(n):
    units = float(input("Enter water consumption: "))

    if units < 0:
        print("Invalid consumption")
        continue

    if units <= 100:
        bill = units * 2
    elif units <= 200:
        bill = 200 + (units - 100) * 3
    else:
        bill = 500 + (units - 200) * 5

    bill += 50

    if units > 250:
        bill += 100
        print("Excessive water usage")

    print("Household:", i + 1)
    print("Total water bill: Rs.", round(bill, 2))
```

8. Test Cases

- 80 units: Bill ₹210.
- 150 units: Bill ₹400.
- 300 units: Bill ₹1150 including penalty.

- Negative units: Invalid consumption.

9. Sample Output

```text
Enter number of households: 2
Enter water consumption: 80
Household: 1
Total water bill: Rs. 210.0
Enter water consumption: 300
Excessive water usage
Household: 2
Total water bill: Rs. 1150.0
```

10. Conclusion

The program calculates water bills using progressive tariff slabs and identifies households with excessive water consumption.

CASE STUDY 36: REPEATED WORD AND PHRASE DETECTION

1. Problem Statement

Process a paragraph, normalize letter case, count word occurrences using permitted string and list operations, and identify repeated words and their positions.

2. Objectives

- Normalize text using lowercase conversion.
- Identify repeated words.

- Count word occurrences.
- Display positions of repeated words.

3. Assumptions

- Words are separated by spaces or punctuation.
- Common punctuation is removed.
- Positions are counted starting from 1.

4. Algorithm

1. Read a paragraph.
2. Convert it to lowercase.
3. Remove common punctuation.
4. Split the paragraph into words.
5. Count each word.
6. Display repeated words and their positions.

5. Flowchart

START → INPUT PARAGRAPH → CONVERT TO LOWERCASE → REMOVE PUNCTUATION → SPLIT INTO WORDS → COUNT OCCURRENCES → DISPLAY REPEATED WORDS AND POSITIONS → END

6. Pseudocode

```
START
INPUT paragraph
CONVERT paragraph to lowercase
REMOVE punctuation
words = SPLIT paragraph
FOR each unique word:
    FIND positions of word
    IF count > 1:
```

        DISPLAY word, count and positions
END

7. Python Source Code

```python
paragraph = input("Enter a paragraph: ").lower()

for mark in ".,!?;:":
    paragraph = paragraph.replace(mark, " ")

words = paragraph.split()
processed = []

for i in range(len(words)):
    word = words[i]

    if word not in processed:
        processed.append(word)
        positions = []

        for j in range(len(words)):
            if words[j] == word:
                positions.append(j + 1)

        if len(positions) > 1:
            print("Word:", word)
            print("Count:", len(positions))
            print("Positions:", positions)
```

```

## 8. Test Cases

- "Python is fun and Python is easy": Python and is are repeated.

- "I like coding": No repeated words.

- "Hello hello world": Hello occurs twice.

## 9. Sample Output

```text

Enter a paragraph: Python is fun and Python is easy

Word: python

Count: 2

Positions: [1, 5]

Word: is

Count: 2

Positions: [2, 6]

```

10. Conclusion

The program uses strings, lists, and loops to detect repeated words and display their positions.

CASE STUDY 45: WEEKLY ELECTRICITY CONSUMPTION MATRIX

1. Problem Statement

Process appliance-wise electricity readings across seven days, calculate daily and appliance totals, and identify consumption exceeding specified limits.
```

2. Objectives

- Store electricity readings in nested lists.
- Calculate appliance-wise weekly totals.
- Calculate daily totals.
- Identify consumption exceeding limits.

3. Assumptions

- Three appliances are considered.
- Seven days of sample readings are used.
- Readings are measured in units.
- Daily limit is 7 units for this demonstration.

4. Algorithm

1. Store appliance names and readings.
2. Calculate weekly consumption for each appliance.
3. Calculate total consumption for each day.
4. Compare daily totals with the limit.
5. Display the results.

5. Flowchart

START → STORE READINGS → CALCULATE APPLIANCE TOTALS → CALCULATE DAILY TOTALS → CHECK LIMIT → DISPLAY RESULTS → END

6. Pseudocode

```
START
STORE appliance names and readings
FOR each appliance:
    CALCULATE weekly total
```

```
    DISPLAY total
FOR each day:
    CALCULATE daily total
    IF total > limit:
        DISPLAY excessive consumption
    DISPLAY daily total
END
```

7. Python Source Code

```python
appliances = ["Fan", "Light", "Computer"]
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

usage = [
    [2, 2, 2, 2, 2, 3, 3],
    [1, 1, 1, 1, 1, 2, 2],
    [4, 3, 4, 4, 5, 2, 1]
]

limit = 7

for i in range(len(appliances)):
    print(appliances[i],
          "weekly total:", sum(usage[i]))

for j in range(len(days)):
    total = 0
```

```
    for i in range(len(appliances)):
        total += usage[i][j]

    print(days[j], "total:", total)

    if total > limit:
        print("Consumption exceeds limit")
```

8. Test Cases

- Monday consumption: 7 units.
- Friday consumption: 8 units.
- Friday should be identified as exceeding the limit.

9. Sample Output

```text
Fan weekly total: 16
Light weekly total: 9
Computer weekly total: 23
Mon total: 7
Tue total: 6
Wed total: 7
Thu total: 7
Fri total: 8
Consumption exceeds limit
Sat total: 7
Sun total: 6
```

10. Conclusion

The program uses nested lists and loops to calculate appliance-wise and daily electricity consumption and identify excessive usage.

CASE STUDY 51: BUS PASSENGER OCCUPANCY SIMULATION

1. Problem Statement

Process boarding and alighting counts across multiple stops, calculate passenger counts at each stop, and flag any capacity violations.

2. Objectives

- Track passenger movement across stops.
- Calculate passenger occupancy.
- Check bus capacity.
- Identify capacity violations.

3. Assumptions

- Initial passenger count is zero.
- Bus capacity is 40 passengers.
- Boarding and alighting values are sample data.
- Alighting cannot exceed the number of passengers already inside the bus.

4. Algorithm

1. Initialize passenger count to zero.
2. Store boarding and alighting values.
3. Process each stop.
4. Subtract alighting passengers.

5. Add boarding passengers.

6. Check bus capacity.

7. Display passenger count and warnings.

5. Flowchart

START → INITIALIZE PASSENGERS → READ STOP DATA → SUBTRACT ALIGHTING → ADD BOARDING → CHECK CAPACITY → DISPLAY OCCUPANCY → END

6. Pseudocode

START

passengers = 0

capacity = 40

FOR each stop:

    INPUT boarding and alighting

    IF alighting > passengers:

        PRINT invalid alighting

    ELSE:

        passengers = passengers - alighting + boarding

    IF passengers > capacity:

        PRINT capacity violation

    DISPLAY passengers

END

7. Python Source Code

```python
capacity = 40

passengers = 0
```

```
boarding = [15, 20, 10, 5, 8]
alighting = [0, 5, 10, 8, 5]

for i in range(len(boarding)):
    print("Stop", i + 1)

    if alighting[i] > passengers:
        print("Invalid alighting count")
        continue

    passengers = passengers - alighting[i]
    passengers = passengers + boarding[i]

    print("Passengers:", passengers)

    if passengers > capacity:
        print("Capacity violation")
    else:
        print("Within bus capacity")
```

8. Test Cases
- After Stop 1: 15 passengers.
- After Stop 2: 30 passengers.
- After Stop 5: 30 passengers.
- Passenger count above 40 should trigger a capacity warning.

9. Sample Output

```text
Stop 1
Passengers: 15
Within bus capacity
Stop 2
Passengers: 30
Within bus capacity
Stop 3
Passengers: 30
Within bus capacity
Stop 4
Passengers: 27
Within bus capacity
Stop 5
Passengers: 30
Within bus capacity
```

10. Conclusion

The program uses lists and loops to simulate passenger boarding and alighting and monitor bus capacity.

CASE STUDY 63: MULTI-PRODUCT PRICE FLUCTUATION ANALYSIS

1. Problem Statement

Process historical product prices stored in lists, calculate percentage changes, identify maximum fluctuations, and flag products exceeding a supplied threshold.

2. Objectives

- Store product prices in lists.
- Calculate percentage changes.
- Identify maximum price fluctuations.
- Flag products exceeding a threshold.

3. Assumptions

- Each product has an old price and a new price.
- Prices are positive values.
- The fluctuation threshold is 10%.
- Both price increases and decreases are considered.

4. Algorithm

1. Store product names and prices.
2. Calculate percentage change for each product.
3. Compare changes with the threshold.
4. Identify the maximum fluctuation.
5. Display the results.

5. Flowchart

START → STORE PRODUCT PRICES → CALCULATE PERCENTAGE CHANGE → CHECK THRESHOLD → FIND MAXIMUM FLUCTUATION → DISPLAY RESULTS → END

6. Pseudocode

START

STORE products and prices

threshold = 10

maximum_change = 0

FOR each product:

```
    change = (new price - old price) / old price * 100
    DISPLAY change
    IF absolute change > threshold:
        DISPLAY warning
    IF absolute change > maximum_change:
        UPDATE maximum change and product
DISPLAY maximum fluctuation
END
```

7. Python Source Code

```python
products = ["Rice", "Oil", "Sugar", "Milk"]

old_prices = [50, 120, 40, 30]
new_prices = [55, 100, 48, 33]

threshold = 10
maximum_change = 0
maximum_product = ""

for i in range(len(products)):
    change = ((new_prices[i] - old_prices[i])
              / old_prices[i]) * 100

    print(products[i], "Change:",
          round(change, 2), "%")

    if abs(change) > threshold:
```

```
        print("Price fluctuation exceeds threshold")

    if abs(change) > maximum_change:
        maximum_change = abs(change)
        maximum_product = products[i]

print("Maximum fluctuation:", maximum_product)
print("Percentage:", round(maximum_change, 2), "%")
```

8. Test Cases

- Rice: ₹50 to ₹55, change is 10%.
- Oil: ₹120 to ₹100, change is approximately -16.67%.
- Sugar: ₹40 to ₹48, change is 20%.
- Sugar has the maximum fluctuation.

9. Sample Output

```text
Rice Change: 10.0 %
Oil Change: -16.67 %
Price fluctuation exceeds threshold
Sugar Change: 20.0 %
Price fluctuation exceeds threshold
Milk Change: 10.0 %
Maximum fluctuation: Sugar
Percentage: 20.0 %
```

10. Conclusion

The program uses lists and arithmetic operations to analyse product price changes and identify significant fluctuations.

CASE STUDY 72: SMART HOSTEL RESOURCE MONITORING

1. Problem Statement

Analyse room-wise electricity and water consumption, calculate charges, identify threshold violations, and compare resource utilization across floors.

2. Objectives

- Monitor electricity and water usage.
- Calculate room-wise charges.
- Identify excessive consumption.
- Compare resource utilization across floors.

3. Assumptions

- Sample data for four hostel rooms is used.
- Electricity costs ₹8 per unit.
- Water costs ₹2 per unit.
- Electricity usage above 100 units is flagged.
- Water usage above 200 units is flagged.

4. Algorithm

1. Store room numbers and floor details.
2. Store electricity and water consumption.
3. Calculate electricity charges.
4. Calculate water charges.

5. Calculate total charges.

6. Check consumption limits.

7. Display room-wise results.

5. Flowchart

START → STORE ROOM DATA → CALCULATE ELECTRICITY CHARGE → CALCULATE WATER CHARGE → CHECK LIMITS → DISPLAY ROOM REPORT → END

6. Pseudocode

START

STORE rooms, floors, electricity and water

FOR each room:

    electricity_charge = electricity * 8

    water_charge = water * 2

    total = electricity_charge + water_charge

    IF electricity > 100:

        DISPLAY electricity warning

    IF water > 200:

        DISPLAY water warning

    DISPLAY total

END

7. Python Source Code

```python
rooms = ["101", "102", "201", "202"]

floors = [1, 1, 2, 2]

electricity = [80, 120, 95, 150]
```

```
water = [150, 180, 220, 250]

for i in range(len(rooms)):
    e_charge = electricity[i] * 8
    w_charge = water[i] * 2
    total = e_charge + w_charge

    print("Room:", rooms[i])
    print("Floor:", floors[i])
    print("Electricity charge:", e_charge)
    print("Water charge:", w_charge)
    print("Total charge:", total)

    if electricity[i] > 100:
        print("High electricity usage")

    if water[i] > 200:
        print("High water usage")
```

8. Test Cases

- Room 101: Total charge ₹940.
- Room 201: Water consumption of 220 units triggers a warning.
- Room 202: Both electricity and water usage trigger warnings.

9. Sample Output

```text
Room: 101
```

Floor: 1

Electricity charge: 640

Water charge: 300

Total charge: 940

Room: 102

Floor: 1

Electricity charge: 960

Water charge: 360

Total charge: 1320

High electricity usage

Room: 201

Floor: 2

Electricity charge: 760

Water charge: 440

Total charge: 1200

High water usage

Room: 202

Floor: 2

Electricity charge: 1200

Water charge: 500

Total charge: 1700

High electricity usage

High water usage

```

## 10. Conclusion
```

The program monitors hostel electricity and water consumption, calculates charges, and identifies rooms with excessive resource usage.

CASE STUDY 84: ENVIRONMENTAL MONITORING DATA ANALYZER

1. Problem Statement

Analyse nested lists of air-quality or water-quality readings, calculate averages and extremes, and classify observations using supplied threshold values.

2. Objectives

- Store environmental readings in nested lists.
- Calculate average readings.
- Identify minimum and maximum values.
- Classify observations using thresholds.

3. Assumptions

- Sample readings for three locations are used.
- Readings are numerical values.
- A reading above 100 is classified as High.
- A reading of 100 or below is classified as Normal.

4. Algorithm

1. Store location names and readings.
2. Calculate the average for each location.
3. Find minimum and maximum readings.
4. Compare readings with the threshold.
5. Display the classification.

5. Flowchart

START → STORE ENVIRONMENTAL READINGS → CALCULATE AVERAGES → FIND MINIMUM AND MAXIMUM → CHECK THRESHOLD → DISPLAY CLASSIFICATION → END

6. Pseudocode

START

STORE locations and nested readings

FOR each location:

    average = sum(readings) / number of readings

    minimum = minimum reading

    maximum = maximum reading

    DISPLAY average, minimum and maximum

    FOR each reading:

        IF reading > threshold:

            DISPLAY High

        ELSE:

            DISPLAY Normal

END

7. Python Source Code

```python
locations = ["Area A", "Area B", "Area C"]

readings = [
    [80, 90, 100],
    [110, 120, 130],
    [60, 70, 75]
]
```

```
threshold = 100

for i in range(len(locations)):
    values = readings[i]

    average = sum(values) / len(values)

    print("Location:", locations[i])
    print("Average:", round(average, 2))
    print("Minimum:", min(values))
    print("Maximum:", max(values))

    for value in values:
        if value > threshold:
            print(value, "- High")
        else:
            print(value, "- Normal")
```

8. Test Cases
- Area A: Average is 90.
- Area B: All readings are above 100.
- Area C: All readings are below 100.

9. Sample Output

```text
Location: Area A
```

Average: 90.0

Minimum: 80

Maximum: 100

80 - Normal

90 - Normal

100 - Normal

Location: Area B

Average: 120.0

Minimum: 110

Maximum: 130

110 - High

120 - High

130 - High

Location: Area C

Average: 68.33

Minimum: 60

Maximum: 75

60 - Normal

70 - Normal

75 - Normal

```

## 10. Conclusion

The program uses nested lists and statistical operations to analyse environmental readings and classify observations using a threshold.

---
```

CASE STUDY 93: SMART PUBLIC TRANSPORT FARE RECONCILIATION

1. Problem Statement

Calculate passenger fares across multiple routes and passenger categories, reconcile collected revenue with expected revenue, and identify discrepancies.

2. Objectives

- Calculate fares for different passenger categories.
- Process multiple routes.
- Calculate expected revenue.
- Compare expected and collected revenue.
- Identify revenue discrepancies.

3. Assumptions

- Sample route distances are used.
- Adult fare is ₹2 per kilometre.
- Children receive a 50% discount.
- Senior passengers receive a 25% discount.
- Collected revenue is entered by the user.

4. Algorithm

1. Store route names and distances.
2. Read passenger counts for each category.
3. Calculate the fare for adults, children, and senior passengers.
4. Calculate expected revenue.
5. Read collected revenue.
6. Compare expected and collected revenue.
7. Display any discrepancy.

5. Flowchart

START → STORE ROUTE DETAILS → INPUT PASSENGER COUNTS → CALCULATE CATEGORY FARES → CALCULATE EXPECTED REVENUE → COMPARE COLLECTED REVENUE → DISPLAY DISCREPANCY → END

6. Pseudocode

```
START
base_rate = 2
STORE route distances
FOR each route:
    INPUT adult_count, child_count, senior_count
    adult_fare = distance * base_rate
    child_fare = adult_fare * 0.5
    senior_fare = adult_fare * 0.75
    expected = adult_count * adult_fare
            + child_count * child_fare
            + senior_count * senior_fare
    INPUT collected_revenue
    discrepancy = expected - collected_revenue
    DISPLAY expected and collected revenue
    IF discrepancy = 0:
        DISPLAY reconciled
    ELSE:
        DISPLAY discrepancy
END
```

7. Python Source Code

```python
```

```
routes = {
    "Route A": 10,
    "Route B": 20,
    "Route C": 15
}

rate = 2

for route, distance in routes.items():
    print("Route:", route)

    adults = int(input("Adult passengers: "))
    children = int(input("Child passengers: "))
    seniors = int(input("Senior passengers: "))

    adult_fare = distance * rate
    child_fare = adult_fare * 0.5
    senior_fare = adult_fare * 0.75

    expected = (
        adults * adult_fare
        + children * child_fare
        + seniors * senior_fare
    )

    collected = float(input("Collected revenue: "))

    discrepancy = expected - collected
```

```
    print("Expected revenue:", expected)
    print("Collected revenue:", collected)

    if discrepancy == 0:
        print("Revenue reconciled")
    else:
        print("Revenue discrepancy:", discrepancy)
```

8. Test Cases

- Route A: 2 adults, 2 children, and 1 senior passenger. Expected revenue is ₹70.
- Collected revenue of ₹70: Revenue reconciled.
- Collected revenue of ₹60: Discrepancy of ₹10.

9. Sample Output

```text
Route: Route A
Adult passengers: 2
Child passengers: 2
Senior passengers: 1
Collected revenue: 60
Expected revenue: 70.0
Collected revenue: 60.0
Revenue discrepancy: 10.0
```

10. Conclusion

The program calculates public transport fares and compares expected revenue with collected revenue to identify discrepancies.

OVERALL CONCLUSION

These ten additional Python case studies demonstrate the practical application of computational thinking and Python programming. They cover variables, data types, operators, conditional statements, loops, strings, lists, nested lists, dictionaries, and calculations.

The programs apply Python concepts to real-world problems such as fuel comparison, traffic penalties, water billing, repeated word detection, electricity monitoring, bus passenger occupancy, product price analysis, hostel resource monitoring, environmental analysis, and public transport fare reconciliation.

END OF ADDITIONAL CASE STUDIES